Guía 1: Lista de usuarios

Agregar y eliminar usuarios con JavaFX + FXML

Programación Orientada a Objetos · DUOC UC
JavaFX 21 · JDK 21 · Maven 3.9

1. Enunciado

Construye una aplicación de escritorio con JavaFX que permita administrar una **lista de usuarios**. La interfaz debe estar hecha con **FXML** (vista) y un **controlador** (lógica), siguiendo lo visto en la guía de FXML y Scene Builder.

Objetivos

- Aplicar la separación vista / controlador / modelo.
- Modelar una entidad con una clase propia, respetando el encapsulamiento.
- Mostrar y modificar una colección de objetos en una interfaz gráfica.
- Validar datos ingresados por el usuario y entregar mensajes claros.

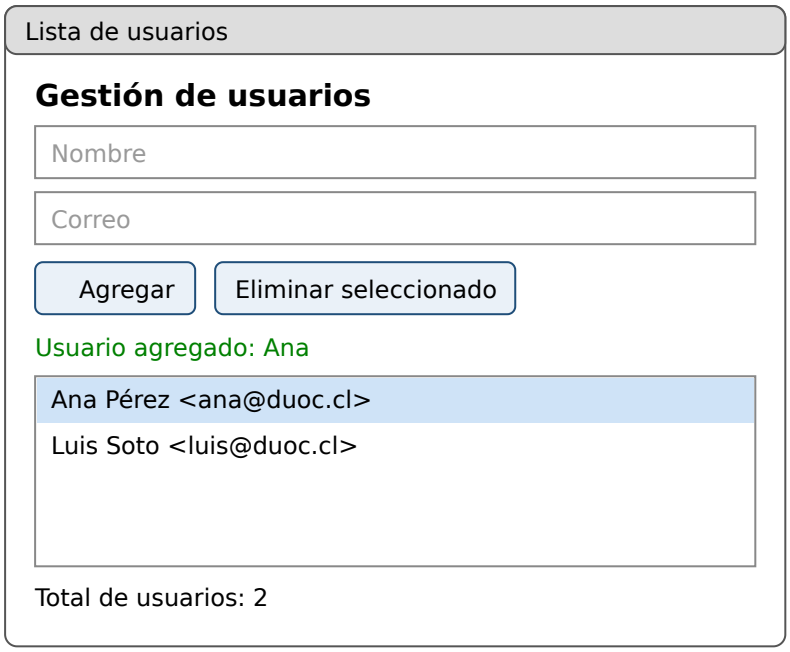
Requisitos funcionales

1. El usuario ingresa un **nombre** y un **correo** y presiona *Agregar*: el usuario aparece en la lista.
2. El usuario selecciona un elemento de la lista y presiona *Eliminar seleccionado*: el elemento desaparece.
3. Si falta el nombre o el correo, se muestra un mensaje de error y **no** se agrega.
4. El correo debe contener `@`.
5. No se pueden repetir correos (sin distinguir mayúsculas de minúsculas).
6. El botón *Eliminar* está **deshabilitado** mientras no haya un elemento seleccionado.
7. Se muestra siempre el total de usuarios, y se actualiza al agregar o eliminar.
8. Después de agregar, los campos se limpian y el cursor vuelve al campo de nombre.
9. Los mensajes de éxito y de error se distinguen visualmente (por ejemplo, por color).

Requisitos de diseño (POO)

- Una clase `Usuario` (modelo) con atributos **privados**, constructor y *getters*.
- `Usuario` debe sobrescribir `equals`, `hashCode` y `toString`, de modo que dos usuarios con el mismo correo se consideren iguales.
- La vista (`.fxml`) **no** contiene lógica; el controlador **no** construye la interfaz.
- Los campos y métodos del controlador accedidos desde la vista deben ser **privados** y anotados con `@FXML`.
- Proyecto Maven con Java 21 y JavaFX 21, que se ejecute con `mvn javafx:run`.

Maqueta de la ventana



Esquema referencial. Puedes ordenar los elementos de otra forma si cumples los requisitos.

2. Qué necesitas investigar

Estos elementos de JavaFX te serán útiles. Revisa la documentación oficial (`openjfx.io`) y las guías anteriores.

Elemento	Pista
`ListView<T>`	Control que muestra una lista de elementos y permite seleccionar uno.
`ObservableList<T>` y `FXCollections`	Una lista que avisa cuando cambia. ¿Cómo se conecta a un `ListView`?
`getSelectionModel()`	¿Cómo saber qué elemento está seleccionado, y qué ocurre si no hay ninguno?
Propiedades y *binding*	¿Cómo hacer que un botón se deshabilite según una condición, sin escribir lógica repetida?
`equals` y `hashCode`	¿Qué método de las listas usa `equals` para saber si un elemento ya existe?
`initialize()`	Método del controlador que se ejecuta cuando la vista ya fue cargada.
`requestFocus()`	Para devolver el cursor a un campo.

Nombres sugeridos

Para que el proyecto sea fácil de revisar, usa estos nombres:

Elemento	Nombre
Paquete	`com.duoc.javafxusuarios`
Clases	`Usuario`, `UsuariosApplication`, `UsuariosController`
Vista	`usuarios-view.fxml`
Campos de texto	`nombreField`, `correoField`
Lista	`usuariosListView`
Botón eliminar	`eliminarButton`
Etiquetas	`mensajeLabel`, `contadorLabel`
Métodos de eventos	`onAgregarClick`, `onEliminarClick`

3. Casos de prueba

Tu aplicación debe comportarse así:

Caso	Acción	Resultado esperado
1	Abrir la app	Lista vacía, “Total de usuarios: 0”, botón *Eliminar* deshabilitado.
2	Presionar *Agregar* con campos vacíos	Mensaje de error; no se agrega.
3	Nombre “Ana”, correo “ana.duoc.cl”	Error: el correo debe contener '@'.
4	Nombre “Ana”, correo “ana@duoc.cl”	Aparece en la lista; mensaje de éxito; campos vacíos; total 1.
5	Repetir con correo “ANA@duoc.cl”	Error: ya existe un usuario con ese correo.
6	Seleccionar “Ana” en la lista	El botón *Eliminar* se habilita.
7	Presionar *Eliminar seleccionado*	Desaparece; total 0; el botón vuelve a deshabilitarse.
8	Escribir un correo y presionar Enter	Se agrega igual que con el botón.

4. Desafíos extra (opcionales)

1. **Confirmar antes de eliminar** con un `Alert` de confirmación.
2. **Editar** un usuario: cargar sus datos en los campos y agregar un botón *Actualizar*.
3. **Buscador** que filtre la lista mientras se escribe.
4. **Validar el correo con una expresión regular**.
5. **Cambiar a `TableView`** con columnas Nombre y Correo.
6. **Agregar más datos** al usuario (edad o rol con un `ComboBox`).
7. **Guardar y cargar** la lista desde un archivo.

5. Entrega y evaluación

Qué entregar

- La carpeta del proyecto Maven completa (sin la carpeta `target/`).
- Debe compilar con `mvn clean compile` y ejecutarse con `mvn javafx:run`.

Checklist

✔	Ítem
☐	El proyecto compila y se ejecuta con Maven.
☐	Existe la clase `Usuario` con atributos privados, *getters*, `equals`, `hashCode` y `toString`.
☐	La interfaz está en un archivo `.fxml`, sin lógica en él.
☐	El controlador usa `@FXML` y `initialize()`.
☐	Se puede agregar y eliminar usuarios.
☐	Hay validaciones: vacíos, formato de correo y duplicados.
☐	El botón *Eliminar* se deshabilita sin selección.
☐	El contador se actualiza al agregar y eliminar.
☐	Nombres claros y código ordenado, con comentarios donde corresponde.